FULL-STACK AI TRACK · DEEP DIVE DD9

Tests that find real bugs

AI-Assisted Testing & Debugging Lab

Participant Handbook · read before Friday

When	Friday 6 November 2026, 08:00 to 11:00 IST (moved from Saturday 7 November)
Format	A hands-on **milestone lab**, mostly on your own: success criteria, hints and checkpoints, not a follow-along. No prompts are handed out. The facilitator checks in twice, after milestone 4 and after milestone 8.
Where	Virtual (Zoom or Teams), working on the Linux lab VM with VS Code and Claude Code
Before you come	About 50 to 60 minutes of reading, plus a 15-minute warm-up. **Nothing to install.** The lab kit is handed out at the start of the session.
Keep it open	Part 5 (the terminal handbook) and the one-page cheat sheet at the back sit next to your terminal on the day

PwC Emerging Business · upGrad Enterprise · Full-Stack AI Track · Delivery material

Contents

1. Start here
2. Part 1 · The session at a glance
3. Part 2 · Before you arrive
4. Part 3 · The ideas
5. Part 4 · How the lab works
6. Part 5 · The terminal handbook
7. Part 6 · The kit, mapped
8. Part 7 · Glossary
9. Part 8 · Answers, reading and the cheat sheet
10. One-page cheat sheet

If you only have 30 minutes: read **Part 1**, the five **"In plain English"** boxes in Part 3, and Part 5, sections 5.1 to 5.6.

Start here

> **Date change.** DD9 has moved from Saturday 7 November to **Friday 6 November, 08:00 to 11:00 IST**. Everything else in this handbook is as planned.

What this handbook is for

On Friday 6 November you will spend three hours finding out what a factory's inspection tests do **not** test, writing the tests that would have caught the bugs, and fixing a real bug with evidence that the fix is right. It is a hands-on session, and it is different from a follow-along: **nobody will give you prompts to paste, and nobody will stop the room after every step.** You will get goals, success criteria and a clock, and you will work out how to get there yourself, with Claude Code, by hand, or both.

That only works if you arrive knowing three things, and this handbook gives you all three:

1. **The ideas** (Part 3): why coverage is a diagnostic and not a target, how to list edge cases before you write any test, how to tell a bug in the code from a mistake in a test, the debugging loop, and how to prove that a test is strong.
2. **The method** (Part 4): how a milestone works, how the hints and the two check-ins work, and how to brief Claude Code well when you are asking it for tests.
3. **The commands** (Part 5): every command you will type, what it does, why we run it, and how to read what it prints. A lot of them are git.

How long it takes. About 50 to 60 minutes to read properly, plus 15 minutes for the warm-up and the self-check in Part 2. If you have only 30 minutes, read Part 1 (the shape of the day), the five "In plain English" boxes in Part 3, and Part 5.1 to 5.6.

What you do not need to do. You do not need to install anything beforehand. The lab kit is handed out at the start of the day, and a minute later it is running. It has no dependencies, so there is not even an `npm ci`. This handbook needs nothing but a reader.

Keep it open on the day. Part 5 (the terminal handbook) and the one-page cheat sheet at the back are made to sit next to your terminal while you work.

What you will be able to do afterwards

By the end of the session you will be able to:

- measure a module's test coverage, and read it as a list of risks instead of a score;
- list the edge cases of a function from its signature and its specification, before any test exists;
- ask an AI for tests, run them, and judge each failure: a bug in the code, or a mistake in the test;
- reproduce a failure with the smallest input, form a hypothesis, and confirm it with a log;
- fix the cause of a bug (not the symptom) and prove the fix with a regression test that failed first;
- report a coverage change by the specific risk it closed, and check with mutation testing that your tests would notice the next bug.

The last four are habits more than facts. They are the point of doing this as a lab.

Where this fits

This is DD9 of 33 sessions in the Full-Stack AI track. On **Monday** (DD7) you refactored a legacy function and learned that a green test run only tells you about what someone thought to test. On **Wednesday** (DD8) you wrote a spec, which is where "what to test" comes from. Today you do the part in the middle: you write the tests, and you find out how good they are. The test suite you build today is one of the pieces you will bring to the **capstone**, so keep your folder. Next comes **DD10** (Wednesday 11 November) on evaluating AI features, which applies the same "prove it can fail" habit to AI behaviour.

Part 1 · The session at a glance

The scenario

Halden Precision, a made-up factory, machines parts and measures every one of them at a gauge station. A small module decides what to do with each reading: the part passes, it is too big and can be machined down (rework), it is too small and cannot be fixed (scrap), or the reading was not usable and the part must be measured again (recheck). The module ships with a handful of tests that the quality team wrote by hand, and a lot of logic that no test has ever asked a hard question. Writing edge cases by hand kept getting pushed back under deadline pressure.

Your job over the morning is to use AI to close that gap in a way that **targets real risk**. You will measure what the tests cover, list the edge cases from the specification, generate tests and judge the failures, find and fix real bugs with a disciplined debugging loop, and then check whether your tests are strong enough to notice the next bug. Everything in the project is made up: the factory, the parts and the numbers.

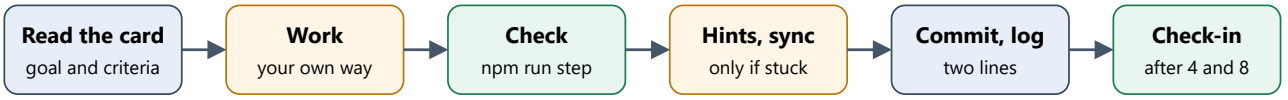
The agenda (Friday 6 November, 08:00 to 11:00 IST)

Time	What happens
08:00	**Kick-off.** The scenario, how the morning works, the ground rules
08:05	**Get the kit.** You receive a link, save a zip, and run four commands. It takes a minute while we carry on
08:09	**Concept sprint.** Coverage as a diagnostic, edge-case prompting, the debugging loop, regression tests, mutation testing. A question first, then the answer
08:22	**Live demo.** Property-based tests find a silent bug in the parser that reads the gauge's text, worked as the method you will use
08:32	Break
08:39	**Lab briefing.** How the cards, the hints and the check-ins work
08:43	**Block A: milestones 1 to 4, on your own.** Baseline the coverage, list the edge cases, generate tests and triage the failures, reproduce a real bug
09:28	**Check-in 1.** The facilitator walks through the steps, the tests and the solutions for milestones 1 to 4
09:43	Break
09:49	**Block B: milestones 5 to 8, on your own.** Instrument and confirm, fix the cause with a regression test, report the delta, run a mutation check
10:28	**Check-in 2.** The same, for milestones 5 to 8
10:44	**Wrap-up.** Reading the output, tools, a short exit quiz, the homework, the close
11:00	End

About two thirds of the session is the lab, and most of that is you working by yourself. The talking is kept short because this handbook did the explaining.

How the lab works

Each of the eight milestones is written like a ticket in a real engineering team. It tells you the **goal**, why it matters, what you are **given**, the **interface contract** (the exact file names, headings and shapes that other documents and the checks depend on), and the **success criteria**. It does **not** tell you how to do it.



then the next milestone, from known-good files

Every milestone follows the same loop. The room only stops twice.

- **Read the card.** The success criteria are your acceptance criteria. Read the card before you touch the keyboard.
- **Work.** You decide how. Briefing Claude Code is a good way; so is doing it by hand. What matters is the result.
- **Check.** `npm run step -- 4` tells you which success criteria are not met yet. You never have to ask anyone whether you are done.
- **Hints and sync, only if you are stuck.** Three hints per milestone, which open a few minutes after you start it. You take them yourself with a command (`npm run hint -- 4`). And `npm run checkpoint -- 4` gives you the reference files, so the next milestone starts from a known-good place. Your own work is kept on a git branch first.
- **Commit and log.** A git commit for each document or change that is ready, and two or three lines in your own evidence log: what you tried, what you learned.
- **Check-in.** After milestone 4 and after milestone 8 the facilitator walks through the steps, the tests and the solutions, and shows the reference brief (the "correct prompt") next to what you wrote.

One thing is different this time: some of your tests are meant to fail. In milestone 3 you will write tests that fail, because they have found real bugs. `npm test` shows red from then until milestone 6 fixes the code. That is the lab working, not you breaking something.

How "done" is measured. A milestone is done when its check says **PASSED** (`npm run step -- 4` prints `STEP 4 PASSED`); `npm run progress` checks them all. Nobody grades your log. The session ends with a five-question exit quiz and a feedback form. A milestone reached by syncing still counts as understood **if you can explain it**.

Part 2 · Before you arrive

Readiness checklist

Tick these before Friday morning. None of them needs the lab kit.

- ☐ You can log in to the lab VM and open **VS Code** on it.
- ☐ The **Claude Code** extension is installed and you are **logged in**. Open the panel and say hello to check.
- ☐ In a terminal on the VM, `node --version` prints **v20 or higher**.
- ☐ In the same terminal, `git --version` prints a version. **This lab uses git all the time.**
- ☐ You know how to open a terminal in VS Code (Terminal menu, then New Terminal) and how to type `cd` and `ls` (Part 5.2).
- ☐ You have a folder where you can save a zip file, such as your home folder.
- ☐ You have read at least Part 1 and Part 5.1 to 5.6 of this handbook.
- ☐ You have done the warm-up below, or tried.

If any of the first four fail, tell the course coordinator **before** the session. They are environment problems, and we can fix them faster in advance than on the day.

Warm-up (about 15 minutes, nothing to install)

You only need a terminal with `node` and `git`. Three small exercises: looking at an old version of a file, a sentence turned into edge cases, and a test that passes while it hides a bug.

Exercise 1 · git, and looking at the past

Two git moves matter today that you may not have used: **reading a file as it was in an earlier commit**, and **putting an old version of one file back**. Make a toy folder:

```
mkdir toy1 && cd toy1
git init
printf 'Gauge notes\n\nShafts are 25.00 mm, plus or minus 0.05.\n' > notes.txt
git add notes.txt
git commit -m "First notes"
```

If git says it does not know who you are, run `git config user.email "you@example.com"` and `git config user.name "Your Name"` and commit again.

Now add a line, commit it, read the file as it was before, and put the old version back after a bad edit:

```
printf 'Shafts on the limit are rejected.\n' >> notes.txt
git diff
git commit -am "Add the limit rule"
git log --oneline
git show HEAD~1:notes.txt
printf 'Oops, a bad edit.\n' > notes.txt
git checkout HEAD~1 -- notes.txt
cat notes.txt
```

```
$ git diff
@@ -1,3 +1,4 @@
 Gauge notes

 Shafts are 25.00 mm, plus or minus 0.05.
+Shafts on the limit are rejected.
$ git log --oneline
64d43eb Add the limit rule
5150954 First notes
$ git show HEAD~1:notes.txt
Gauge notes
Shafts are 25.00 mm, plus or minus 0.05.
$ git checkout HEAD~1 -- notes.txt
$ cat notes.txt
Gauge notes
Shafts are 25.00 mm, plus or minus 0.05.
$ git status --short
M  notes.txt
```

(On Windows git may also print a line about LF and CRLF. It is harmless. Your commit ids will differ.) `HEAD~1` means "one commit before the latest". `git show HEAD~1:notes.txt` prints the file as it was then, without changing anything. `git checkout HEAD~1 -- notes.txt` puts that version into your folder, and the `M` in `git status --short` says it is now a change waiting to be committed. In milestone 6 you will read the old code this way.

Exercise 2 · From a sentence to edge cases (pencil and paper)

Here is one sentence from a made-up drawing: **"A hole passes inspection if its diameter is 10.00 mm, plus or minus 0.5."** List every input you would want to test. Do not write any code. Look at the sentence carefully. What happens at exactly 10.50? At 9.50? At 10.49? What if the gauge sends nothing? What if it sends a negative number, or 0? What if the number has more decimals than the gauge can measure? Aim for at least eight cases, and next to each write the answer you think is right. Compare yours with Part 8.

Exercise 3 · A test that passes, and hides a bug

A test that runs a line is not the same as a test that checks it. Make a second toy folder:

```
mkdir toy2 && cd toy2
echo '{"type":"module"}' > package.json
```

Create `hole.js`:

```
export function holePasses(size) {
  return Math.abs(size - 10) <= 0.5;
}
```

Create `hole.test.js`:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { holePasses } from './hole.js';

test('a hole of 10.00 passes', () => {
  assert.equal(holePasses(10), true);
});

test('a hole of 11.00 fails', () => {
  assert.equal(holePasses(11), false);
});
```

Run it with the coverage report turned on:

```
node --test --experimental-test-coverage hole.test.js
```

```
$ node --test --experimental-test-coverage hole.test.js
✓ a hole of 10.00 passes (1.4148ms)
✓ a hole of 11.00 fails (0.6775ms)
ℹ tests 2
ℹ pass 2
ℹ fail 0
ℹ start of coverage report
ℹ file      | line % | branch % | funcs % | uncovered lines
ℹ hole.js   | 100.00 |   100.00 |  100.00 |
ℹ end of coverage report
```

100% coverage, and everything green. Now add a test for the boundary, a hole of exactly 10.50, which the drawing says is **on the limit and so not accepted**:

```
test('a hole of exactly 10.50 is on the limit, so it fails', () => {
  assert.equal(holePasses(10.5), false);
});
```

```
$ node --test hole.test.js
✓ a hole of 10.00 passes (0.6953ms)
✓ a hole of 11.00 fails (0.1209ms)
✖ a hole of exactly 10.50 is on the limit, so it fails (0.7909ms)
ℹ tests 3
ℹ pass 2
ℹ fail 1
...
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:
  true !== false
```

The 100% was real, and it hid a bug. Now fix the function (change `<=` to `<`) and run again: all three go green. Finally change `<` back to `<=` and run once more. Which test goes red? That is what "a test that can fail" means, and it is what the mutation check at the end of the lab does for you automatically.

Self-check (10 minutes)

Answer in a sentence each, then compare with Part 8.

1. A module has 40% coverage. What does that tell you, and what does it not tell you?
2. Why list the edge cases *before* asking an AI to write the tests?
3. An AI writes a test that fails. Name two different explanations. How do you tell them apart?
4. What is the difference between reproducing a failure and fixing it?
5. A hypothesis says "the comparison is wrong". Why is that not good enough?
6. Why must a regression test fail before the fix?
7. You change `25 + 0.05` to compare as whole thousandths of a millimetre. What problem is that solving?
8. Coverage says every line ran. A mutation check says one change to a comparison survives. What does that mean?

Part 3 · The ideas

The session leans on nine ideas. Each section ends with an **In plain English** box: if you remember nothing else, remember those. None of the lab's answers is in this chapter. The examples use a made-up bakery oven and a made-up parcel scale, not the factory in the lab.

3.1 Coverage is a diagnostic, not a target

A coverage tool runs your tests and records which lines of the code ran, and which branches (the two sides of every `if`). The result is a percentage for each file. The number is easy to read and easy to misuse, so be clear about what it says.

- **It says which code no test has ever run.** That is a list of questions nobody has asked.
- **It does not say which of those matter.** An untested line that changes the words on a screen and an untested line that decides whether a part is scrapped look the same to the tool.
- **It cannot see code that is not there.** If a function has no branch for a missing reading, there is nothing to leave uncovered.
- **It can be raised without making anything safer.** A test that calls a function and checks nothing runs every line.

Lines and branches can also disagree. A function can have most of its lines run and only a few of its branches, because the happy path runs most of the lines once and nothing ever takes the other side of the `if`.

> **In plain English.** 40% coverage does not mean the code is 40% safe. It means 60% of it has never been asked a hard question. Read the gaps as a list of risks, and rank them by what a wrong answer would cost.

3.2 Edge cases come before tests

If you ask an AI "write tests for this function", it writes tests for the cases it thinks of, and so would you. Those are the cases that needed no help. The bugs live in the others. So do it in two steps: **first list the edge cases, then write the tests.** The list is where you can check your own and the AI's assumptions, cheaply, before any test exists.

Four families cover most of what goes wrong:

Family	Examples (an oven that must hold 180 °C, plus or minus 5)
The happy path	180, 178, 183
Boundaries	exactly 175, exactly 185, and one step either side of each
Invalid input	nothing at all, `NaN`, a negative number, a number far too big to be real, a number written as text
Ordering and combinations	two readings arriving in the wrong order; a rule that depends on the type of oven

For a boundary, test **exactly at it, just above it, and just below it**. And ask what the specification says *at* the line: is "within 5 degrees" inclusive or exclusive? That one word decides whether a reading of exactly 185 is a pass.

> **In plain English.** Write the list before the tests. Boundaries and bad input are where the bugs hide, and an AI will skip them unless you ask.

3.3 Tests come from the specification, not from the code

If you give an AI only the code and ask for tests, it reads what the code does and writes tests that agree with it. A bug in the code becomes a passing test. The test suite then protects the bug.

The fix is to give the AI the **specification** as well, and to say that the specification wins. In this lab the specification is `docs/tolerance-spec.md`. When a test and the code disagree, the question is never "which one looks right to me?" but "what does the specification say?"

> **In plain English.** A test written from the code asks the code if it agrees with itself. A test written from the specification asks it if it is right.

3.4 A failing test is a question, not a verdict

When an AI-generated test fails there are three explanations: the **code** is wrong, the **test** is wrong (the AI invented a rule), or the **specification** is unclear. The two lazy answers are both dangerous: "the test must be wrong, the code has worked for years" hides a real bug, and "the code must be wrong" can send you to change correct code.

So you triage. For each failing test, find the sentence of the specification that decides it, and write down the verdict and the section. If the specification agrees with the test, the code is wrong. If it says something else, fix the test. If it says nothing, that is a question for the owner of the specification, and you do not decide it alone.

> **In plain English.** Before you decide who is wrong, read the specification. Write down which sentence settled it.

3.5 Reproduce, hypothesize, instrument, confirm

Debugging by guessing is slow and it fixes the wrong thing. A better loop has four steps, in this order:

1. **Reproduce** the failure, with the smallest input you can find (one reading, one part), and find the nearest input that does *not* fail. Where the two meet is where the bug starts.
2. **Hypothesize.** Write one specific cause: where it is, and what you think happens. Then write what you would expect to see in a log if you are right, and what would show that you are wrong. A hypothesis nothing can disprove is a guess with better vocabulary.
3. **Instrument.** Add one targeted log line that prints exactly the values the hypothesis is about, and run the failing input. Do not print everything.

4. **Confirm.** Read the output. It confirms the hypothesis, rejects it, or revises it. Then take the log out.

The most useful outcome is often the third one. A hypothesis that is right about one case and only half right about the module tells you there is more than one cause, before you have "fixed" it.

> **In plain English.** Smallest failing input, a guess that a log could disprove, one log line, and read what it says.

3.6 Fix the cause, not the symptom

A fix that makes today's failing test pass is not necessarily a fix. Two ways to get it wrong:

- **Special-casing.** `if (reading === 1.65) return 'rework'` makes one test green and leaves every other value broken.
- **A half-fix.** You change the comparison and the one example you had goes green, but the cause was bigger than the comparison.

One cause deserves a mention because it surprises people: **computers add decimals badly.** Try it:

```
$ node -e "console.log(0.1 + 0.2, 25 + 0.05, 1.6 + 0.05, 1.6 + 0.05 === 1.65)"
0.30000000000000004 25.05 1.6500000000000001 false
```

`25 + 0.05` happens to come out as exactly `25.05`, and `1.6 + 0.05` does not come out as `1.65`. A rule such as "a part on the limit is rejected" works for one size and fails for another, depending on the digits. The usual cure is to compare whole numbers: if the gauge measures to 0.001, turn every size into whole thousandths (round the size times 1000) and compare those.

> **In plain English.** A fix that works for one example may not work for the next. Ask why the example failed, and fix that, for every case.

3.7 Red first: the regression test

A **regression test** guards a bug that was fixed, so that it cannot quietly come back. It only proves something if you saw it **fail before the fix** and **pass after**. A test written after the fix and never seen red might be checking something that was always true.

The sequence is short: write the test, run it and watch it fail on the old code, make the fix, run it again and watch it pass. In the lab you will do this with git: take the old version of the code from an earlier commit and run your tests against it.

> **In plain English.** If the test never failed, you do not know that it can.

3.8 Do your tests notice? Mutation testing and property tests

Coverage tells you a line ran. It does not tell you a test would notice if the line were wrong. A **mutation check** answers that: it makes a copy of the code with one small deliberate bug (a `<` turned into `<=`, a `+` into a `-`, a `'scrap'` into a `'rework'`), runs your tests, and counts how many tests fail. A bug that no test fails on is a **surviving mutant**: your tests visit the line but would pass even if its logic were wrong.

Some survivors are real gaps (write the test that kills them). Some are **equivalent**: the change cannot alter any answer, so no test can notice it. Part of the skill is telling them apart.

A **property-based test** attacks the same problem from the other side. Instead of a few hand-picked inputs, you state something that must be true for *every* input ("any text either parses to a number or throws a clear error, never `NaN`"), and a helper tries hundreds of generated inputs and shows you the smallest one that breaks it. You will see one in the demo.

> **In plain English.** A test that runs a line but would pass even if the line were wrong is not testing the line, it is visiting it.

3.9 Report the delta by the risk it closed

"Coverage went up from 70% to 100%" convinces nobody. "These tests now catch a part exactly on its limit being accepted, and a missing reading being scrapped" answers the real question: what is safer now? A credible report has four parts: the before and after numbers, how they were measured, the risks closed (each tied to a test that exists), and what is **still not covered**. That last part is what makes the rest believable.

> **In plain English.** Name the risk, name the test, and say what you did not do.

Part 4 · How the lab works

4.1 Anatomy of a milestone card

Every milestone is a card in `LAB-GUIDE.md`, written like a ticket. Here is one for a made-up task that is **not** part of the lab, so you can see what each section is for.

Milestone X · List the cases for a parcel scale *(an example, not a real milestone)*

Ticket SHOP-000 · aim for **8 minutes** · hints open at **+3, +5, +7** *The pace to aim for, and when each hint opens (counted from when you start the milestone).*

Why this matters. Nobody has written down what the scale should do at its limits, so every bug is a surprise. *The reason, so you can judge trade-offs.*

Your goal. A short list of the cases the scale's function must handle, each with the answer the rules give. *One outcome, in one sentence.*

You are given. The scale's rules, the function, and Claude Code. *Everything you may use, so you never have to guess what exists.*

Interface contract. The file is `docs/scale-cases.md` : a table with the columns Case, Input and Expected, and at least ten rows. Commit it. *The exact names, files and headings that other documents and the checks depend on. These are requirements, and they are not negotiable.*

Success criteria.

- **Must:** ☐ the table has ten rows or more ☐ each limit is covered ☐ the answers come from the rules, not the code ☐ it is committed ☐ `npm run step -- X` passes
- **Stretch:** ☐ list the cases the rules do not settle, with a question for each *How you will know you are done. "Must" is the finish line. "Stretch" is for when you are early.*

Where to start. What happens exactly at the limit? What if the scale sends nothing at all? *Questions to settle before you type. They are not instructions.*

Verify yourself. `npm run step -- X` *The command that tells you which criteria are not met.*

The things a card does **not** contain: a prompt to paste, a solution, or the names of the traps. Finding those is the milestone.

4.2 Working mostly on your own

This lab is built so that you can work at your own pace, without anyone stopping the room after every step.

Block	Milestones	About how long	Then
A	1, 2, 3, 4	45 minutes	**Check-in 1:** the facilitator walks through the steps, the tests and the solutions for 1 to 4
B	5, 6, 7, 8	39 minutes	**Check-in 2:** the same, for 5 to 8

- **The pace on each card is a guide, not a deadline.** Some milestones will take you less, some more. A slow milestone costs you nothing, because syncing is free and the next milestone starts from known-good files.
- **You do not need permission to move on.** When `npm run step -- N` says PASSED, go on to the next milestone.
- **Pace marks in the chat.** At a few set times the facilitator posts a line such as "by now, aim to be starting milestone 3". It is there so that you can see how you are doing against the room. You do not need to reply.
- **At the check-ins** the facilitator shows the reference solutions and the reference briefs, and compares them with what you built. They may ask you to paste your `npm run progress` line in the chat (see 4.8).
- **If you finish a block early,** do the stretch goals on the cards. They are the best use of spare time. Or help a neighbour.

4.3 The hint ladder

Each milestone has **three hints** and then a **reveal**. They are meant to be taken in order, each only when you need it.

Level	What it gives you	Example (for the made-up milestone above)
Hint 1: Nudge	Where to look, or the right question to ask	"What happens exactly at the limit? What does the rule say about it?"
Hint 2: Approach	The shape of the solution, without the answer	"Ask the AI to read the rules as well as the code, and to say where they disagree."
Hint 3: Near-solution	Almost the answer, enough to finish	"A limit is a case of its own: test exactly at it, just above it and just below it."
Reveal	The reference brief (a good prompt), the approach, and the common mistakes	Shown by the facilitator at the check-ins

How the clock works: it starts for a milestone the first time you run `npm run start -- 4`, `npm run step -- 4` or `npm run hint -- 4`. Each hint opens a few minutes after that. If you ask too early, the command tells you how long to wait:

```
Milestone 1 · Find the drift
Hint 1 of 3 opens in about 5 minutes (the clock for this milestone started 0 minutes ago).
Try the questions under "Where to start" on the card first. That is where most of the learning is.
If you really need it now:  npm run hint -- 1 --now   (it is noted as an early hint, and you can
say so in LAB-LOG.md)
```

That is a nudge, not a lock. `--now` always works, and the early hint is counted in your `npm run progress` table.

How to use hints well:

- **Try first.** A few minutes of being stuck is where most of the learning happens.
- **Take them with the command:** `npm run hint -- 4` gives the next one for milestone 4. `npm run hint -- 4 --again` re-reads the ones you have taken.

- **If you take one early, write it in your log.** That is not a penalty: it is how you will remember what you needed.
- **The reveal** is `npm run hint -- 4 --reveal`. It is meant for the check-ins, when the facilitator shows it. Reading it early is allowed, and honest logging is all that is asked.

4.4 Syncing: catching up for free

If you are not where you want to be, you do not fall behind:

```
npm run checkpoint -- 4
```

This replaces your work with the reference solution for milestone 4, so milestone 5 starts from known-good files. Because this lab is built on git, it does this **with real commits**: it goes back to the starting commit and replays the reference solution one step at a time, so the history looks like the work of a careful engineer, with one commit for each document or change. That matters, because the history is part of what the checks look at.

Your own work is kept. Everything you had committed, and anything you had not committed yet, goes onto a branch called `my-work-<time>`. `git branch` lists it, and `git log my-work-<time> --oneline` shows what was on it.

Using a checkpoint is not cheating. It is the reference solution, and the next milestone is built on it. What matters is that you understand it before you move on: read it, and write one line in your log about what it does differently from your version. You can sync at any time, not only at the check-ins.

4.5 Working with Claude Code on tests

Claude Code is a very fast, very confident colleague. It reads the folder you open, follows the rules in `CLAUDE.md`, runs commands, and edits files. It is also sometimes wrong, and it will not always tell you. When you ask it for **tests**, its mistakes are quiet: a test that passes because it agrees with a bug, a list of cases that skips the awkward ones, a fix that makes today's example green. Your job is not to type; it is to **decide what is true, and check that the tests say it**.

Writing a brief

Because there is no prompt to paste, the card's success criteria become the acceptance criteria in **your** brief. A good brief has five parts:

Part	What to say	Example
Context	What exists, and what it is for	"This is a Node project. The specification is `docs/tolerance-spec.md` and the code is `src/inspection/classifyDefect.js`."
Task	One thing to do, in one sentence	"List the edge cases for `classifyDefect`. Do not write any tests yet."
Constraints	What it must not do	"Take the expected answers from the specification, not from the code. Say where the code disagrees. Do not edit `src/`."

Part	What to say	Example
Acceptance	The observable results that mean "done": headings, names, what a run prints	"A table with the columns Case, Input, Expected and Why, and at least 14 rows."
Verify	The commands or checks that prove it	"Then I will run `npm run step -- 2`."

Five habits that matter more than any wording

1. **Give it the specification, not only the code.** Tests written from the code assert the code.
2. **List first, test second.** Ask for the cases, check the list, then ask for the tests.
3. **Ask it to say where it is unsure.** An AI that has to name its guesses makes fewer of them silently.
4. **Make it quote.** "For each failing test, quote the sentence of the specification that decides it." Then check three quotes yourself.
5. **Run it, don't believe it.** "The tests pass" is a claim. The output is the evidence.

When Claude is confidently wrong

It will sometimes write a test that asserts a bug, invent a rule that the specification does not contain, or "fix" a failure by special-casing the one input it was shown. None of that is malice; it is what a fluent writer does with an ambiguous source. The defences are the ones in Part 3: the specification wins, the smallest failing input, a log that could disprove the hypothesis, a regression test that failed first, and a mutation check.

Two safety rules

1. **Read the diff before you commit.** In this lab that means every file Claude touched, not only the one you asked about.
2. **Never paste a real key, password or customer's data into Claude or the chat.** Everything in the kit is made up, so you do not need to.

4.6 Reading a failing check

When `npm run step -- N` fails it prints which success criteria are not met yet. Here is a real one, from milestone 1, run before any document existed:

```
MILESTONE 1: Baseline the coverage

✕ docs/baseline.md exists with a table of coverage numbers and a table of untested gaps (1.9825ms)
✕ the coverage numbers are what the tool prints for the team's tests (0.3796ms)
✕ every block of classifyDefect.js that the team's tests never run has a row, with its real line
numbers (0.2972ms)
...
✕ failing tests:

✕ docs/baseline.md exists with a table of coverage numbers and a table of untested gaps (1.9825ms)
  AssertionError [ERR_ASSERTION]: docs/baseline.md is missing. Create it with the two tables from
the milestone card.

STEP 1 not passing yet. Read the first failure above: it says which success criterion is not met.
Stuck? npm run hint -- 1 gives you the next hint when it opens. To catch up, npm run checkpoint --
1 gives you the reference.
```

- **The ✓ and ✕ lines** are the individual criteria. Fix them from the top.
- `pass` **and** `fail` are counts. You want `fail 0`.
- **The message after** `AssertionError` is the useful sentence. It says what is wrong, and for a document it says what the check was looking for. It never prints the answer: for the edge-case list it will not tell you the right answer for a row.
- **For code, the check runs your function** on a few thousand readings and compares it with a hidden answer key. If it differs, it shows you an input and what it should have said. For tests, it runs them against the code as shipped, against the correct code, and against slightly wrong versions, and tells you which wrong versions your tests did not notice.
- **The last two lines** tell you what to do next.

A failing check is information, not a verdict. Most are fixed by reading the message slowly.

4.7 The evidence log

`LAB-LOG.md` is your own record. After each milestone, write two or three lines: how you did it (your brief, or by hand), whether you took a hint (and whether it was early), and what you learned or what surprised you. Anything that looks wrong but is not part of the milestone goes under "Noticed". Nobody marks it. It is how you remember the lesson on Friday, and it is the habit the capstone's delivery narrative will ask for.

4.8 Asking for help, and the pulse

- **Say what you are stuck on.** "Stuck on 6: my fix passes for the shaft but the check says another part is still wrong" gets a better answer than "it's broken".
- **Teaching assistants** watch the chat and answer in one line. They will not give you an answer, but they will ask the question that gets you to it. They will also help with anything about your machine.
- **The pulse.** Shortly before each check-in you may be asked to run `npm run progress` and paste the **last line** in the chat. It looks like this:

```
PULSE M1[x] M2[x] M3[x] M4[ ] M5[ ] M6[ ] M7[ ] M8[ ] hints=2
```

It tells the facilitator where the room is, and nobody else can see your name against a result. It takes a few seconds to a minute to run, because it runs the check for every milestone.

Part 5 · The terminal handbook

Every command you will type on Friday, what it does, why we run it, and how to read what it prints. The lab VM runs Linux, and every command here is written for its shell (bash). The examples show real output from the lab kit, so what you see on the day will look the same, apart from things like dates, commit ids, timings and file paths.

Each entry has the same shape:

- **What it does**, in plain English
- **Why we run it** in this lab
- **How to read the output**
- **If it goes wrong**

5.1 How to read a command

A command is a program, then what to do, then the details:

```
npm run step -- 4
|   |   |    |  └─ an argument: the number 4
|   |   |    └──── "--": everything after this goes to the script, not to npm
|   |   └───────── the name of a script defined in package.json
|   └───────────── a subcommand: run a script
└───────────────── the program
```

Piece	Looks like	Meaning
Program	`npm`, `node`, `git`	The thing being run
Subcommand	`run`, `commit`, `show`	What to ask the program to do
Flag	`--grep`, `--team`, `--now`	Changes how it behaves. Some take a value
Argument	`4`, `src/inspection/classifyDefect.js`	What to act on
Comment	`# like this`	Ignored by the shell: it is there for you

Four things that catch people out:

1. **Quotes matter.** Inside **single quotes** (`'...'`) everything is literal. Inside **double quotes** (`"..."`) the shell replaces `$VARIABLES` with their values. A commit message with spaces needs quotes: `git commit -m "Add the edge cases"`.
2. **A backslash at the end of a line** (`\`) means "this command carries on on the next line". It must be the last character on the line, with no space after it.
3. **Copy-paste can change characters.** Word processors and chat tools turn straight quotes into curly ones, and two dashes into one long dash. Either breaks a command. If a pasted command fails mysteriously, retype the quotes and dashes.
4. **Case and spaces matter.** `npm run step -- 4` is not `npm run step --4`.

> **Tip: tab completion.** Press **Tab** while typing a file or folder name and the shell finishes it for you. Press it twice to see the options.

5.2 Moving around and looking at files

Command	What it does	Why we run it
`pwd`	Prints the folder you are in	To check you are in `inspection-service`, not one level up (the most common mistake)
`ls -A`	Lists the files in the folder, including hidden ones (names starting with a dot)	To see what is there. `-A` shows hidden files such as `.gitignore`
`cd dd9-lab/inspection-service`	Moves into a folder. `cd ..` moves up one level, `cd ~` goes home	To get to the right place before every npm command
`cat docs/tolerance-spec.md`	Prints a file's contents	To read a small file without opening an editor
`head -n 20 file`	Prints the first 20 lines of a file or of a command's output	To look at the top of something long
`grep "recheck" src/inspection/classifyDefect.js`	Prints the lines of a file that contain some text	To search from the terminal
Ctrl+C	Stops the program that is running in this terminal	To get out of anything that is stuck
`clear`	Clears the screen	To tidy up

If it goes wrong: `No such file or directory` almost always means you are in the wrong folder. Run `pwd`, then `ls`, then `cd` to where you meant to be.

5.3 Getting the kit

You will be given a link and a block of commands on the day. Here is what each one does.

```
cd ~
```

Go to your home folder, so the kit goes somewhere predictable.

```
unzip -q dd9-lab-kit.zip
```

Unpack the zip. `-q` means quiet: it does not list every file. If `unzip` is not installed, use `python3 -m zipfile -e dd9-lab-kit.zip .`, which does the same job.

```
cd dd9-lab/inspection-service
```

Move into the folder you will work in. **Only this folder is opened in VS Code.** The folders next to it (`hints` , `checks` , `checkpoints`) hold answers, and Claude Code reads everything in the folder you open.

```
$ ls ..
README.md
checkpoints
checks
hints
inspection-service

$ ls -A
CLAUDE.md
LAB-GUIDE.md
LAB-LOG.md
docs
package.json
scripts
src
tests
```

(`ls -A` also shows a few hidden files that start with a dot.) There is **no `npm ci` step** in this lab: the project has no dependencies, so there is nothing to download. It is Node's own test runner and a few plain scripts.

5.4 Node and npm

Command	What it does	Why we run it
`node --version`	Prints the Node version, for example `v24.14.1`	To check Node is new enough (v20 or higher)
`npm test`	Runs every test in the project	To confirm the starting point is healthy (`pass 10`), and to see what your tests find
`npm test -- --grep shaft`	Runs only the tests whose name contains `shaft`	To run the one test you are investigating
`npm run coverage`	Runs the tests and prints which lines and branches ran	Milestones 1 and 7
`npm run mutate -- <file>`	Puts small bugs into a copy of the file and runs your tests	Milestone 8 and the homework
`npm run <name>`	Runs a script named in `package.json`	Almost everything in the lab is a script: `step` , `hint` , `coverage` ...
`npm run help`	Prints every lab command with a one-line explanation	When you forget one

`npm test`

```
✓ kit: the specification has six numbered sections (4.2447ms)
✓ kit: the part table has seven parts, each with a nominal size and a tolerance (0.4567ms)
✓ kit: the property helper makes the same inputs every run (0.5074ms)
✓ kit: the property helper reports a failing input and makes it smaller (0.9551ms)
✓ a shaft at its nominal size passes (1.1053ms)
...
ℹ tests 10
ℹ pass 10
ℹ fail 0
```

How to read it: each `✓` is a test that passed and each `✕` one that failed. The last block is the summary: you want `fail 0` (but see below). There are three families of tests: `tests/baseline` are the lab's own sanity tests, `tests/existing` are the quality team's tests of the shipped code, and `tests/learner` are yours (there are none until milestone 3).

This lab is different: from milestone 3 until milestone 6, `npm test` shows red, on purpose. Your tests have found bugs. For example:

```
ℹ tests 29
ℹ pass 20
ℹ fail 9
✕ a NaN reading asks for a recheck
✕ a shaft exactly on its upper limit is reworked
✕ every part exactly on its lower limit is scrapped
...
```

The double dash

```
npm test -- --grep shaft
```

What `--` does: it tells npm "everything after this is for the script, not for you". Without it, npm would try to read `--grep` itself. You will use it for `npm test`, `npm run step`, `npm run hint` and `npm run coverage`.

Running only some tests: `--grep`

```
npm test -- --grep "upper limit"
```

```
✓ .../tests/baseline/kit.test.js
✓ .../tests/existing/inspection.test.js
✕ a shaft exactly on its upper limit is reworked (2.3742ms)
✓ a shaft just inside its upper limit passes (0.1306ms)
✓ a shaft just over its upper limit is reworked (0.1239ms)
✕ every part exactly on its upper limit is reworked (0.3997ms)
ℹ tests 6
ℹ pass 4
ℹ fail 2

✕ failing tests:
✕ a shaft exactly on its upper limit is reworked (2.3742ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

  'pass' !== 'rework'
```

How to read it: `--grep` keeps only the tests whose **name** contains the text. The lines that show a file path are test files, which node lists and counts, so `tests 6` here is four real tests and two file lines. Read the failure from the bottom: `'pass' !== 'rework'` means the code returned `'pass'` and the test expected `'rework'`. Which one is right is for the specification to say.

`npm run coverage`

```
npm run coverage -- --team
```

```
Coverage by the team's tests only (tests/existing)

File                              Lines %  Branches %  Never run (line numbers)
src/inspection/classifyDefect.js     70.4        40.0  13-14 16-17 23-26
src/inspection/errors.js             52.9       100.0  5-8 13-16
src/inspection/label.js              60.0        66.7  12-13 18-25
src/inspection/parser.js            100.0        66.7  -
src/inspection/parts.js             100.0       100.0  -

Made with: node --experimental-test-coverage on Node v24.14.1
```

What it does: runs the tests with Node's built-in coverage on, and prints the numbers for each file under `src/inspection/`. **Why we run it:** `-- --team` measures only the quality team's tests (`tests/existing`): that is your **baseline**, and it stays the same however many tests you add. It does change when the code changes, so write the baseline numbers down in milestone 1 (in `docs/baseline.md`): after you fix the code, the "before" in your report is those numbers. Without `--team` it measures all of the tests, which is how you get the "after" number. You can add a file name to look at one file: `npm run coverage -- classifyDefect`.

How to read it: Lines % is the share of lines that at least one test ran. **Branches %** is the share of the two sides of each `if` that ran. **Never run** lists the line numbers no test reached: open the file at those lines and ask what happens there. A `-` means every line ran.

If it goes wrong: "No coverage numbers found" means the tests did not run. Run `npm test` to see why.

`npm run mutate`

```
npm run mutate -- src/inspection/classifyDefect.js --list
```

```
M01  line  15  === -> !==
M02  line  16  recheck -> pass
M03  line  18  + -> -
...
11 mutants in src/inspection/classifyDefect.js
```

What it does: `--list` shows the small deliberate bugs it will put into a copy of the file, one at a time. Without `--list` it runs your tests against each one:

```
KILLED    M01  line  15  === -> !==   (caught by: a shaft at its nominal size passes)
SURVIVED  M02  line  16  recheck -> pass
KILLED    M03  line  18  + -> -   (caught by: a shaft at its nominal size passes)
...
11 mutants: 4 killed, 7 survived. Score 36.4%
```

How to read it: KILLED means a test failed on that bug, which is what you want. **SURVIVED** means every test still passed: no test noticed. Each survivor is a question: what input would make the bugged code give a different answer from the real code? Your files are never changed: it works in a temporary copy. It refuses to run if your tests do not all pass first.

5.5 git, the commands you will use

This lab uses git in every milestone, because the history is evidence, and in milestone 6 you will run your tests against an old version of the code. You need these commands.

Command	What it does	Why we run it
`git status --short`	Lists the files that changed since the last commit	To see what you have changed and not yet committed
`git add -A`	Chooses every change for the next commit	Before a commit
`git commit -m "message"`	Saves the chosen changes as a commit	After each document or change that is ready
`git log --oneline`	The history, newest first, one line per commit	To see your steps, and to find the starting commit at the bottom
`git diff`	Shows exactly what changed since the last commit	To review a change before you commit it
`git show <commit>:<file>`	Prints a file as it was at an earlier commit	To read the shipped code after you have changed it
`git checkout <commit> -- <file>`	Puts one file back as it was at a commit	Milestone 6: to run your tests against the old code
`git branch`	Lists branches	To find the `my-work-...` branch a sync made for you

`git status --short`, `git add`, `git commit`

```
$ git status --short
?? docs/baseline.md
$ git add -A
$ git commit -m "Add the coverage baseline"
```

How to read it: `??` means a new file git does not know yet; `M` means a file you changed. After `git add -A` and a commit, `git status --short` prints nothing. Commit after each document or change that is ready, with a message that says what it is. The lab's checks look at your commits.

`git log --oneline`

```
$ git log --oneline
2cc7c6b Edge-case tests from the specification, and the triage of the failures
a6adbeb List the edge cases for classifyDefect from the specification
815828d Baseline the coverage: numbers and ranked gaps
d03c9b7 Shipped code as delivered
```

How to read it: the newest commit is on top, and the **starting commit** (`Shipped code as delivered`, made by `npm run setup`) is at the bottom. Your ids will differ.

`git diff`

```
git diff HEAD~1 --stat
```

```
 docs/triage.md                | 16 ++++++++
 tests/learner/classify.test.js | 93 ++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 109 insertions(+)
```

How to read it: `git diff` alone shows what you changed since the last commit, line by line (`+` added, `-` removed). With `HEAD~1` it compares with the commit before the latest one, and `--stat` summarises by file. **Read the diff before you commit,** especially a diff that an AI wrote: it is how you see the one file it touched that you did not ask about.

`git show` and `git checkout <commit> -- <file>`: running your tests on the old code

You will use these in milestone 6, to prove a regression test fails on the code as it was shipped.

```
git log --oneline                                          # the starting commit is the bottom
one
git show d03c9b7:src/inspection/classifyDefect.js          # READ the old file, change nothing
git checkout d03c9b7 -- src/inspection/classifyDefect.js   # put the old file back in your
folder
npm test -- --grep regression                              # your regression tests should be
red
git checkout HEAD -- src/inspection/classifyDefect.js      # put your fixed file back
npm test -- --grep regression                              # and green
```

How to read it: `d03c9b7` is the starting commit's id (use yours). `git show` only prints; `git checkout <commit> -- <file>` replaces your copy of that one file with the old version, so `git status --short` will show it as changed. **Always finish with `git checkout HEAD -- <file>`,** which brings your latest committed version back. If you had uncommitted work in that file, commit it first.

If it goes wrong: `git commit` saying "Please tell me who you are" means git has no name or email. `npm run setup` sets one for this folder; if it did not, run `git config user.email "you@example.com"` and `git config user.name "Your Name"` inside the project.

5.6 The lab commands

`npm run setup`

```
$ npm run setup
git had no name or email on this machine, so this repository now uses "Lab learner
<learner@lab.local>" for your commits.
Done. The starting commit is d03c9b7634: "Shipped code as delivered".
Everything you change from now on shows up in git status and git diff. Commit after each document
or change that is ready.
```

What it does: turns the folder into a git repository (if it is not one) and makes the **starting commit** that holds the code and the documents exactly as delivered. **Why we run it:** the checks compare your work with that commit. **Run it once, before you change anything:** if the shipped files were already edited it refuses, and you re-extract the zip.

`npm run doctor`

```
$ npm run doctor
  ok    Node 24.14.1
  ok    the shipped code, src/inspection/classifyDefect.js, is there
  ok    the specification, docs/tolerance-spec.md, is there
  ok    checks/, checkpoints/ and hints/ found next to this folder
  ok    git version 2.53.0.windows.2
  ok    the starting commit exists (d03c9b7634)

Ready. Open LAB-GUIDE.md and start milestone 1.
```

What it does: checks Node, git, the layout and the starting commit. **If it prints `FAIL`,** the line under it says what to do.

`npm run start -- N`

```
Milestone 3 · Generate tests, triage failures
The clock for this milestone just started.
Aim to finish in about 14 minutes. Hints open after 7, 10, 13 minutes (a nudge, not a lock: npm run
hint -- 3).
Open LAB-GUIDE.md at the card for this milestone, and read it before you type anything.
```

What it does: starts the clock for milestone N and says when its hints open. It is optional: the first `step` or `hint` command for that milestone starts the clock for you.

`npm run step -- N`

What it does: runs the hidden check for milestone N and says which success criteria are not met. **Why we run it:** it is how you know you are done, without asking anyone. Part 4.6 explains how to read a failure. A pass looks like this:

```
MILESTONE 3: Generate tests, triage failures

✓ there are at least 12 tests in tests/learner, and none is skipped (461.1781ms)
✓ every test agrees with the specification (none of them asserts what the buggy code does)
(0.2232ms)
✓ at least three of your tests fail on the code as shipped: those are the bugs you found (0.133ms)
✓ every failing test has a row in docs/triage.md, with the right verdict (3.3471ms)
✓ each verdict cites the section of the specification that decides it (1.3865ms)
✓ the tests and the triage are committed, and nothing else was changed (59.2544ms)
STEP 3 PASSED
Milestone 3 is done. Write two lines in LAB-LOG.md, commit, then go on to the next milestone.
```

`npm run progress`

```
$ npm run progress
  [x]  1  Baseline the coverage             done  (6 pass, 0 fail)  hints 1 (1 early)
  [x]  2  Enumerate the edge cases          done  (7 pass, 0 fail)  hints 0
  ...
  [ ]  7  Report the coverage delta         not yet  (0 pass, 5 fail)  hints 0

6 of 8 milestones done, 1 hints taken.
Next: milestone 7. Run  npm run step -- 7  to see exactly which part is failing, or  npm run hint --
- 7  for a hint.

PASTE THIS IN THE CHAT:
PULSE M1[x] M2[x] M3[x] M4[x] M5[x] M6[x] M7[ ] M8[ ] hints=1
```

What it does: runs the check for every milestone, and prints one line to paste in the chat. **Why we run it:** the facilitator sometimes asks for it shortly before a check-in, to see where the room is. It takes a few seconds to a minute.

`npm run hint -- N`

```
$ npm run hint -- 1
Milestone 1 · Baseline the coverage
Hint 1 of 3 opens in about 5 minutes (the clock for this milestone started 0 minutes ago).
Try the questions under "Where to start" on the card first. That is where most of the learning is.
If you really need it now:  npm run hint -- 1 --now   (it is noted as an early hint, and you can
say so in LAB-LOG.md)
```

What it does: gives you the next hint for milestone N when it has opened, and tells you how long to wait if it has not. `--now` takes it early (and notes that), `--again` re-reads the hints you have taken, and `--reveal` shows the reference brief and the common mistakes.

`npm run checkpoint -- N`

```
$ npm run checkpoint -- 3
Your code now matches the reference solution for milestone 3, in 3 commits on top of the starting commit.
Run `npm test` to confirm, then carry on with the next milestone.
```

What it does: replaces your work with the reference solution for milestone N, as real git commits. If you had work of your own, it first saves it on a branch called `my-work-<time>`; `git branch` lists it. **Why we run it:** to catch up for free. Using it is not cheating. What matters is that you can explain the milestone afterwards, and one sentence in your log is enough.

`npm run help`

Prints every command in the kit on one screen, with a line about each.

5.7 When something goes wrong

What you see	Most likely cause	What to do
`No such file or directory`	You are in the wrong folder	`pwd`, then `cd` to `dd9-lab/inspection-service`
`Missing script: "step"`	You are one folder up	`cd dd9-lab/inspection-service`
`Unsupported engine`, `SyntaxError`	Node is too old	`node --version` must say v20 or higher
`git: command not found`	git is not installed on this VM	Tell the facilitator: this lab needs it
`Please tell me who you are`	git has no name or email	`npm run setup` sets one; or `git config user.email "you@example.com"` and `git config user.name "Your Name"`
A check says "there is no starting commit"	`npm run setup` has not run	Run it (it refuses if the code or the documents were already edited: re-extract the zip)
A check says the files that are not yours were changed	Someone (or Claude) edited `scripts/`, `tests/existing`, the specification or `parts.js`	`git checkout -- <the file>`
A check says uncommitted changes	You have not committed the document or change	`git status --short`, then `git add -A` and `git commit -m "..."`
`npm test` is red after milestone 3	Your tests found bugs: that is the point	Go on. It turns green when the fix in milestone 6 is done
`npm test` fails in `tests/existing`	The behaviour of the team's tests changed	Find the change. Do not edit the test
A check says a test disagrees with the specification	A test asserts what the buggy code does, or an invented rule	Read the section it points at, then fix the test

What you see	Most likely cause	What to do
A check says your tests did not notice a mistake	Your tests would stay green if the code were slightly wrong	Write a test that would fail for that mistake
`npm run mutate` says your tests do not all pass	It needs a green suite to start from	Run `npm test`, fix what fails
Claude edited files it should not (`scripts/`, `tests/existing`)	It ignored `CLAUDE.md`	`git checkout -- scripts tests/existing`, then restate the constraint

Part 6 · The kit, mapped

You get the kit on the day. This is what is in it, so that nothing is a surprise.

The folders

```
dd9-lab/
├── README.md                 setup, in one page
├── inspection-service/       ← the ONLY folder you open in VS Code
│   ├── LAB-GUIDE.md          your guide: the nine milestone cards
│   ├── LAB-LOG.md            your evidence log
│   ├── CLAUDE.md             the project's context file (Claude Code reads it)
│   ├── package.json          the scripts (`npm run ...`); no dependencies
│   ├── docs/                 tolerance-spec.md (the specification), then your own documents
│   ├── src/inspection/       the shipped code: classifyDefect.js, parser.js, sampling.js,
parts.js, label.js, errors.js
│   ├── tests/
│   │   ├── baseline/         the lab's own sanity tests
│   │   ├── existing/         the quality team's tests of the shipped code
│   │   ├── learner/          your tests
│   │   └── lib/              property.js, a tiny property-based testing helper
│   └── scripts/              the lab's runner (step, hint, progress, checkpoint, coverage,
mutate)
├── checks/                   the hidden checks, and the answer key they compare with
├── checkpoints/              the reference solution after each milestone
└── hints/                    the three hints for each milestone, and the reference briefs
```

Why only `inspection-service`? Claude Code reads every file in the folder you open. If you opened `dd9-lab`, it would find the reference solutions, and it would use them. The folders next to `inspection-service` hold the answers on purpose.

What you write, and what you leave alone

Yours	Not yours
`tests/learner/`, new documents in `docs/`, `LAB-LOG.md`, `src/inspection/classifyDefect.js` (from milestone 6), `src/inspection/sampling.js` (the homework)	`scripts/`, `tests/baseline/`, `tests/existing/`, `tests/lib/`, `docs/tolerance-spec.md`, `src/inspection/parts.js`, `package.json`

The specification is **not yours to edit** on purpose. A specification you change to fit the code proves nothing, and the part table belongs to Quality Engineering.

What is real and what stands in

In the kit	In real life
Claude Code	The real thing. There is no API key and no model in the kit
The factory, the parts and the numbers	Made up

In the kit	In real life
`npm run coverage` (Node's built-in coverage)	Istanbul, c8, or your language's own coverage tool
`npm run mutate` (a small mutation tester, about 100 lines)	Stryker, PIT, mutmut: the same idea with many more kinds of bug
`tests/lib/property.js` (a small property-based helper)	fast-check, Hypothesis, jqwik
The hidden checks	A reviewer who reads your tests and your notes

Part 7 · Glossary

Term	Plain-English meaning
Baseline	The numbers before you change anything. Here: the coverage of the team's tests alone
Boundary	The edge of an allowed range. Test exactly on it, just above it and just below it
Branch	One side of an `if`. Branch coverage counts how many sides ran
Checkpoint	The reference solution after a milestone. `npm run checkpoint -- N` puts it in your folder, as git commits
Coverage	The share of lines, or branches, that the tests ran. A diagnostic, not a goal
Edge case	An input at or beyond the edge of what the code expects: a limit, nothing, a bad value
Equivalent mutant	A change to the code that cannot alter any answer, so no test can notice it
Hint ladder	Three hints and then a reveal for each milestone, taken one at a time
Hypothesis	A specific guess about the cause of a bug, written so that a log could prove it wrong
Instrument	Add a targeted log line to see the values the code really uses
Mutant	A copy of the code with one small deliberate bug. A mutant is **killed** if a test fails on it and **survives** if none does
Mutation testing	Running your tests against mutants to find out whether they would notice a bug
Property-based test	A test of something that must be true for every input, run on hundreds of generated inputs
Pulse	The one line `npm run progress` prints for pasting in the chat
Red / green	A test that fails (red) and then passes (green). A test should be seen red first
Reference brief	A good prompt for a milestone, shown at the check-ins
Regression test	A test that guards a fixed bug so that it cannot come back. It must fail before the fix and pass after
Reproduce	Make a failure happen on demand, with the smallest input
Root cause	The thing that actually makes the failure happen, as opposed to the symptom you can see
Sync	Another word for using a checkpoint to catch up
Triage	Deciding, for each failing test, whether the code or the test is wrong, and writing down why

Part 8 · Answers, reading and the cheat sheet

Answers: the warm-up

Exercise 1 (git). `git show HEAD~1:notes.txt` prints the file as it was one commit ago and changes nothing. `git checkout HEAD~1 -- notes.txt` puts that old version into your folder; `git status --short` then shows `M`, a change waiting to be committed. Afterwards `git checkout HEAD -- notes.txt` brings the latest committed version back.

Exercise 2 (a sentence made into edge cases). For "a hole passes if its diameter is 10.00 mm, plus or minus 0.5", a good list has at least:

Case	Input	Expected
in the middle	10.00	pass
just inside the upper limit	10.49	pass
exactly on the upper limit	10.50	depends on the rule: "plus or minus 0.5" does not say. Ask!
just over the upper limit	10.51	fail
just inside the lower limit	9.51	pass
exactly on the lower limit	9.50	the same question
just under the lower limit	9.49	fail
no reading	nothing, `null`, `NaN`	not a pass and not a fail: measure again
zero or negative	0, -3	a sensor fault, not a verdict on the hole
absurdly large	10 000	a sensor fault
too many decimals	10.4999	depends: the gauge cannot measure that

The important finding is the two "depends": the sentence does not say whether the limit itself is allowed. A good tester finds that, and asks the owner of the specification instead of guessing.

Exercise 3 (a test that passes and hides a bug). The 100% coverage was real, and the function was still wrong on the boundary, because no test ever asked what happens exactly on the limit. With `<` the three tests pass. If you change it back to `<=`, the boundary test goes red, and only that one: it is the only test that guards the boundary.

Answers: the self-check

1. It says 60% of the code, or of the branches, has never been run by a test, so nobody has asked it a hard question. It does not say which gaps matter, whether the other 40% is well tested, or what is missing from the code altogether.
2. Because the list is where you can check your own and the AI's assumptions, before any test exists. An AI asked for tests straight away writes tests for the cases it thought of, and skips the awkward ones.

3. The code is wrong, or the test is wrong (the AI invented a rule). You tell them apart by finding the sentence in the specification that decides the case. If the specification agrees with the test, the code is wrong.
4. Reproducing is making the failure happen on demand with the smallest input; it tells you where to look. Fixing is changing the code. If you skip the first, you fix the bug you imagined.
5. Because nothing could show it is wrong. A good hypothesis names a line and a mechanism, and says what a log would show if you are right and if you are wrong.
6. Because a test that has never failed might be checking something that was always true. Seeing it fail on the old code, and pass on the new, is the proof that the fix did something.
7. Floating-point addition: `1.6 + 0.05` is not exactly `1.65`, so a comparison with a limit can go the wrong way for some sizes. Comparing whole thousandths removes the problem.
8. It means a test visits that line but would pass even if the comparison were wrong. Either write a test that kills the mutant, or show that the change cannot alter any answer (an equivalent mutant).

Reading

- For the idea of **mutation testing**: any introduction to a mutation tool for your language (Stryker for JavaScript, PIT for Java). The lab does a small version for you.
- For **property-based testing**: the documentation of fast-check (JavaScript), Hypothesis (Python) or jqwik (Java).
- Earlier in the programme: the DD7 handbook's Part 3 on characterisation tests, and the DD8 handbook's Part 3 on writing a spec. Today's lab is the part between them.

Apply it at work: ten questions for any AI-generated tests

1. Did the AI see the specification, or only the code?
2. Were the edge cases listed before the tests were written?
3. Is there a test exactly on every boundary, and one either side?
4. What happens for no input, a bad input, and an impossible input?
5. When a generated test failed, who checked the specification before deciding who was wrong?
6. Was each new test red before the fix?
7. If I broke the rule slightly, would any test notice? (Try a mutation check, or break it by hand.)
8. Did the fix reach the cause, or only the example that failed?
9. Does the coverage report name the risk each gain closed, or only a percentage?
10. What is still not covered, and does the report say so?

One-page cheat sheet

Where am I? `pwd` (should end in `inspection-service`) · `git status --short` · `git log --oneline`

I want to…	Command
Run all the tests	`npm test`
Run one test, or a family	`npm test -- --grep "upper limit"`
Measure the baseline (the team's tests only)	`npm run coverage -- --team`
Measure everything	`npm run coverage`
See the bugs the mutation check will try	`npm run mutate -- src/inspection/classifyDefect.js --list`
Run the mutation check	`npm run mutate -- src/inspection/classifyDefect.js`
Check milestone N	`npm run step -- N`
Check everything, get the line for the chat	`npm run progress`
Get a hint (early, again, reference)	`npm run hint -- N` (`--now`, `--again`, `--reveal`)
Catch up for free	`npm run checkpoint -- N`
Save a document or change	`git add -A` then `git commit -m "..."`
See what I changed	`git diff`
Read an old version of a file	`git show <commit>:<file>`
Put one file back as it was	`git checkout <commit> -- <file>` (then `git checkout HEAD -- <file>` to undo)
Find the starting commit	`git log --oneline` (the bottom one), or `cat .lab/baseline-commit`

The loop: list the edge cases first · write tests from the specification · for every failure, find the sentence that decides it · reproduce with the smallest input · hypothesize in a way a log could disprove · one targeted log line · fix the cause, for every case · the regression test fails first · report the risk, not the percentage · run the mutation check.

Remember: coverage is a diagnostic, not a target · the specification wins until its owner says otherwise · a test that never failed proves nothing · a fix for one example is not a fix.